Zofia Kapcińska 160227

Sprawozdanie: Implementacja algorytmu Diffiego-Hellmana uzgadniania klucza sesyjnego

1. Screenshot.

```python
import hashlib
import time
import random
import string
import numpy as np

def generateHash(text, algorithm):  4 usages
    hashObj = hashlib.new(algorithm)
    hashObj.update(text)
    return hashObj.hexdigest()

def findCollision(algorithm='sha256'):  1 usage
    hexNum = 12 // 4
    seen = {}
    attemps = 1

    while 1:
        randomText = ''.join(random.choices(string.ascii_letters + string.digits, k=10))
        fullHash = generateHash(randomText.encode('utf-8'), algorithm)
        prefix = fullHash[:hexNum]

        if prefix in seen:
            originalText = seen[prefix]
            originalHash = generateHash(originalText.encode('utf-8'), algorithm)
            print("\nZnaleziono kolizję po " + str(attemps) + " próbach:")
            print("Tekst 1: " + originalText + " | Hash: " + originalHash)
            print("Tekst 2: " + randomText + " | Hash: " + fullHash)
            print("Wspólny prefix: " + str(prefix))
            break

        seen[prefix] = randomText

def toBits(data):  1 usage
    bits = np.unpackbits(np.frombuffer(data, dtype=np.uint8))
    return bits

def sha256Bits(data):  2 usages
    hashObj = hashlib.new('sha256')
    hashObj.update(data)
    return toBits(hashObj.digest())
```

```python
def testSAC_sha256():  1 usage
    outputBytesLength = 256
    changeCounter = np.zeros(outputBytesLength)

    for _ in range(1000):
        inputBytes = np.random.bytes(32)
        hashBits = sha256Bits(inputBytes)

        modifiedInput = bytearray(inputBytes)

        byteID = random.randint( a: 0,  b: 31)
        bitID = random.randint( a: 0,  b: 7)

        modifiedInput[byteID] ^= (1 << bitID)
        modifiedHash = sha256Bits(modifiedInput)
        changeCounter += (hashBits ^ modifiedHash)

    return changeCounter / 1000
```

```python
if __name__ == '__main__':
    algorithms = ['md5', 'sha1', 'sha224', 'sha256', 'sha384', 'sha512', 'sha3_224', 'sha3_256', 'sha3_384', 'sha3_512']
    userText = input('Podaj tekst: ')

    testData = ["Lorem ipsum dolor sit amet, consectetur efficitur.",
                "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras nec sollicitudin arcu. Curabitur sed sodales nunc, vel feugiat lectus. Nullam tristique scelerisque libero eu ultrices. Suspendisse proin.
                "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque consequat mauris sed mi mollis accumsan. Aenean fermentum commodo risus a venenatis. Duis ut elit urna. Praesent in malesuada nisl, et
                "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed tempus gravida dolor, vitae molestie tellus tempus et. Aliquam malesuada posuere enim viverra finibus. Duis elementum felis libero, ut dict

    for algorithm in algorithms:
        start = time.time()
        val = generateHash(userText.encode('utf-8'), algorithm)
        end = time.time()
        print("\nAlgorytm " + algorithm + ": \nTekst użytkownika o długości " + str(len(userText)) + " znaków: " + val + " | Czas działania: " + str(end - start) + " | Długość ciągu wyjściowego: " + str(len(val)))
        for data in testData:
            start = time.time()
            val = generateHash(data.encode('utf-8'), algorithm)
            end = time.time()
            print("Tekst testowy o długości " + str(len(data)) + " znaków: " + val + " | Czas działania: " + str(end - start) + " | Długość ciągu wyjściowego: " + str(len(val)))

    findCollision('sha256')
    prob = testSAC_sha256()
    meanProb = np.mean(prob)
    print("\nŚrednie prawdopodobieństwo zmiany dla wszystkich bitów na wyjściu po zmianie jednego bitu na wejściu: " + str(round(meanProb, 3)))
```

2. **Omówienie sposobu implementacji.**
   Aplikacja została zaimplementowana w języku Python z użyciem bibliotek:
   - hashlib, zawierającej natywny dostęp do algorytmów kryptograficznych
   - time, pozwalającej na zmierzenie czasu działania algorytmów
   - random, pozwalającej na wygenerowanie danych do testów oraz wybranie losowych indeksów
   - string
   - numpy

   Do generowania skrótów użyłam funkcji hashlib.new(), która pozwala na dynamiczny wybór algorytmów.

   W badaniu kolizji wykorzystałam słownik, w którym zapisywane są 12-bitowe prefiksy oraz oraz odpowiadający im losowo wygenerowany tekst. Sprawdzam, czy prefiksy występują wielokrotnie, co oznacza kolizję.

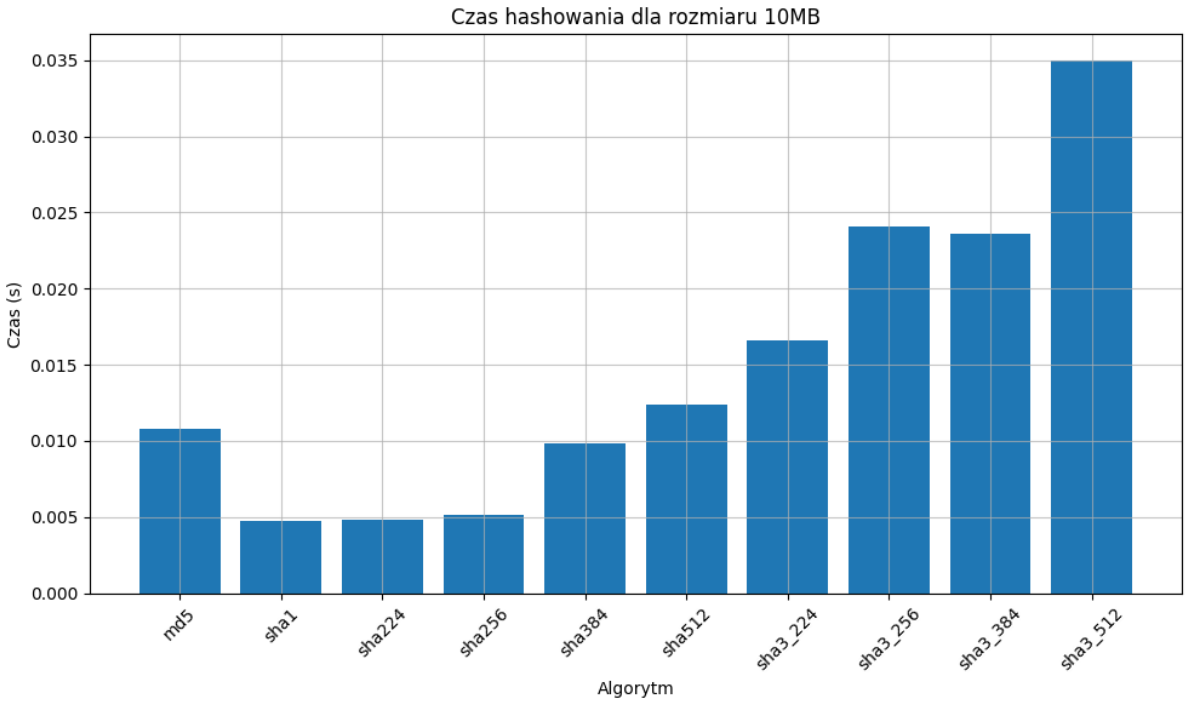
   Do zbadania własności SAC generowane jest 256 bitów oraz funkcja skrótu. Następnie zmieniany jest jeden losowy bit w oryginalnym ciągu (XOR) i ponownie generowany jest skrót. Badana jest zmiana bitów między skrótami z użyciem XOR-a oraz obliczone zostaje prawdopodobieństwo zmiany.

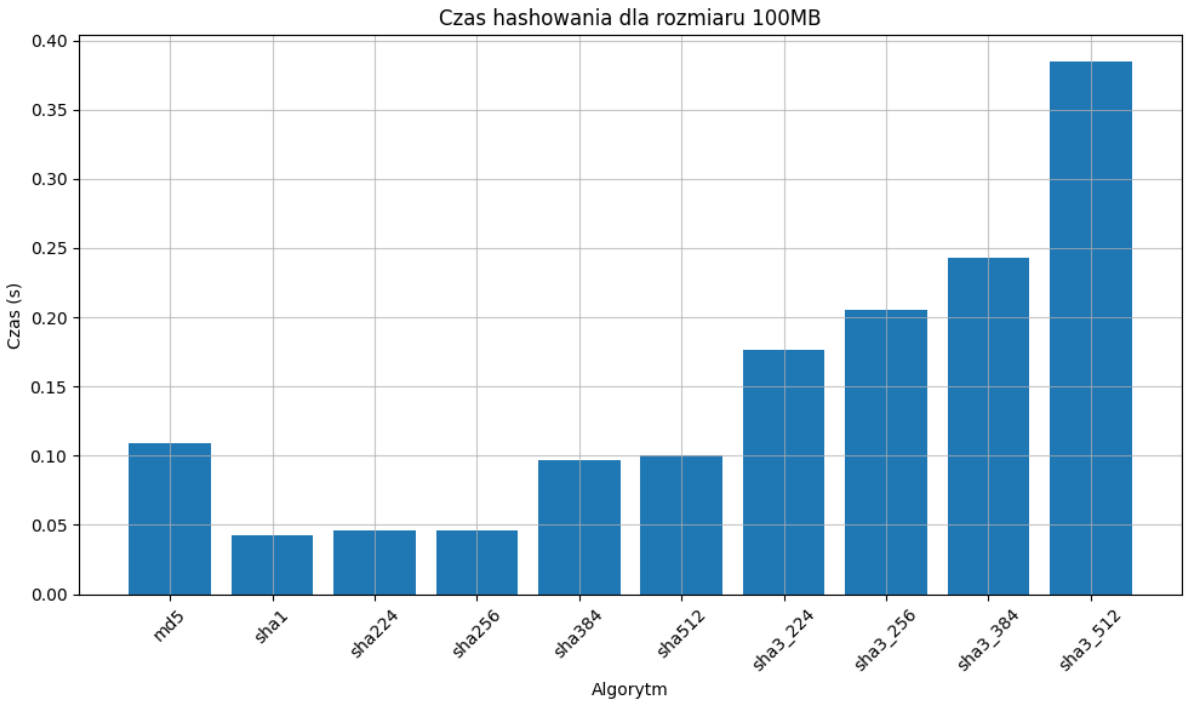
3. **Określenie roli soli w tworzeniu skrótów.**
   Sól to losowy ciąg danych dodawany do hasła przed procesem hashowania. Zapewnia to unikalną zmianę wyniku, dzięki czemu nawet identyczne hasła mają różne skróty. Zwiększa to bezpieczeństwo i zapobiega łamaniu haseł.

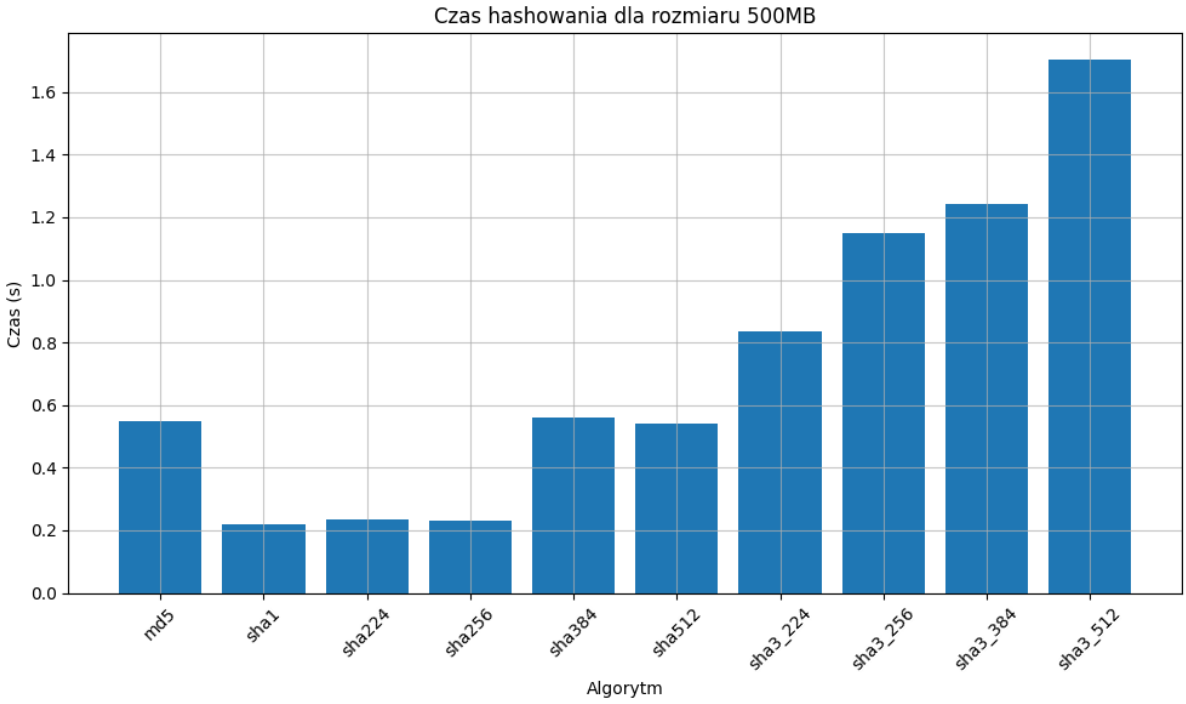
4. **Na podstawie powszechnie dostępnych źródeł odpowiedz na pytanie – czy funkcję MD5 można uznać za bezpieczną? Czy dotychczas zostały znalezione dla niej jakiekolwiek kolizje?**
   Funkcja MD5 nie może być uznana za bezpieczną. Jest mało odporna na ataki oraz podatna na kolizje, które zostały znalezione także w moich testach.

5. **Zestawienie wyników wraz ze stosownymi wnioskami.**
   1) Porównanie algorytmów

Algorytm	**Długość na wyjściu [w bitach]**	**Czas działania (10 MB) [s]**	**Czas działania (100 MB) [s]**	**Czas działania (500 MB) [s]**
MD5	128	0.01084	0.11019	0.55026
SHA-1	160	0.00471	0.04316	0.21801
SHA-2 (224)	224	0.00476	0.04593	0.23296
SHA-2 (256)	256	0.00512	0.04599	0.23507
SHA-2 (384)	384	0.01286	0.09806	0.49419
SHA-2 (512)	512	0.00980	0.09858	0.49447
SHA-3 (224)	224	0.01587	0.15828	0.79466
SHA-3 (256)	256	0.01762	0.17777	0.88169
SHA-3 (384)	384	0.02164	0.21949	1.09639
SHA-3 (512)	512	0.03198	0.31633	1.58012




Czas hashowania dla rozmiaru 100MB
Czas (s)
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
md5
sha1
sha224
sha256
sha384
sha512
sha3_224
sha3_256
sha3_384
sha3_512
Algorytm


Czas hashowania dla rozmiaru 500MB
Czas (s)
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
md5
sha1
sha224
sha256
sha384
sha512
sha3_224
sha3_256
sha3_384
sha3_512
Algorytm

2) Kolizje
   Znaleziono kolizję po 1 próbach:
   Tekst 1: 7G9tmmpGL8
   Hash:
   7be0e08890612812d111c2dda31947151ecb1e418042518a2c7feb478c5c0e7b

   Tekst 2: LF8HZgKRhS
   Hash:
   7be4971570be5267513e421dec7cdaf619f29eaba7f3df0c1266df22559657dd

   Wspólny prefix: 7be

3) SAC
   Średnie prawdopodobieństwo zmiany dla wszystkich bitów na wyjściu po zmianie jednego bitu na wejściu: 0.5

Wnioski:

Niezależnie od długości tekstu wejściowego (od 10 do 500 MB), dany algorytm zawsze generuje skrót o identycznej liczbie znaków. Czas obliczeń rośnie wraz ze wzrostem objętości danych. Algorytmy z rodziny SHA-3 wymagają nieco więcej operacji, co jest szczególnie widoczne przy przetwarzaniu większych bloków danych. Im dłuższy ciąg wyjściowy, tym trudniejsze jest znalezienie kolizji, co bezpośrednio przekłada się na wyższy poziom bezpieczeństwa kryptograficznego